SIMATS ENGINEERING

ASSESSMENT TOOL 3 - Output Prediction

COURSE NAME: Data Structures **COURSE CODE:** CSA0308

Register Number	**192411185**
Name	**CH.BHARATH KUMAR REDDY**

Assessment Tool Weightage: 35%

COURSE OUTCOME COVERED

CO1: *Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL3, BL4)*

Questions: 5 **EACH QUESTIONS CARRIES:** 5 MARKS **Total Marks:** 25 **Duration:** ONE HOUR

S. No.	List of Questions	

1	**Array Traversal Program**	`#include <stdio.h>`
		`int main()`
		`{`
		`  int a[5] = {10, 20, 30, 40, 50};`
		`  int i;`
		`  for(i = 0; i < 5; i++)`
		`  printf("%d ", a[i]);`
		`  return 0;`
		`}`

2	**Sum of Array Elements Program**	`#include <stdio.h>`
		`int main()`
		`{`
		`  int a[] = {2, 4, 6, 8};`
		`  int sum = 0, i;`
		
		`  for(i = 0; i < 4; i++)`
		`  sum += a[i];`
		
		`  printf("%d", sum);`
		`  return 0;`
		`}`

3	**Recursive Factorial Program**	`#include <stdio.h>`
		
		`int fact(int n)`
		`{`
		`  if(n == 0)`
		`  return 1;`
		`  return n * fact(n - 1);`
		`}`
		
		`int main()`
		`{`
		`  printf("%d", fact(4));`
		`  return 0;`
		`}`

4	**Recursive Fibonacci Program**	`#include <stdio.h>`
		
		`int fib(int n)`
		`{`
		`  if(n <= 1)`
		`  return n;`
		`  return fib(n-1) + fib(n-2);`
		`}`
		
		`int main()`
		`{`
		`  printf("%d", fib(6));`
		`  return 0;`
		`}`

5	**Function as ADT Operation Program**	`#include <stdio.h>`
		
		`int add(int x, int y)`
		`{`
		`  return x + y;`
		`}`
		
		`int main()`
		`{`
		`  printf("%d", add(15, 25));`
		`  return 0;`
		`}`

Code of Conduct:

I **CH.BHARATH KUMAR REDDY(192411185)***certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

CH.BHARATH KUMAR REDDY
Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (5 Marks)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (3 Marks)	Level 1 – Beginning (1–2 Marks)
Understanding of Program Logic	1	Correctly identifies the complete program flow and execution sequence.	Minor misunderstanding of program flow.	Partial understanding of logic.	Unable to interpret the program logic.
Trace of Variables / Function Calls	1	Accurately traces all variable values, iterations, and recursive calls.	Traces most values correctly with minor errors.	Traces some values but misses important steps.	Incorrect or no tracing.
Output Prediction Accuracy	2	Output predicted completely and correctly.	Output mostly correct with minor mistakes.	Output partially correct.	Output incorrect or missing.
Complexity Interpretation (Time/Space)	1	Correctly identifies and justifies complexity where applicable.	Correct complexity with limited justification.	Partial understanding of complexity.	Incorrect or no complexity analysis.

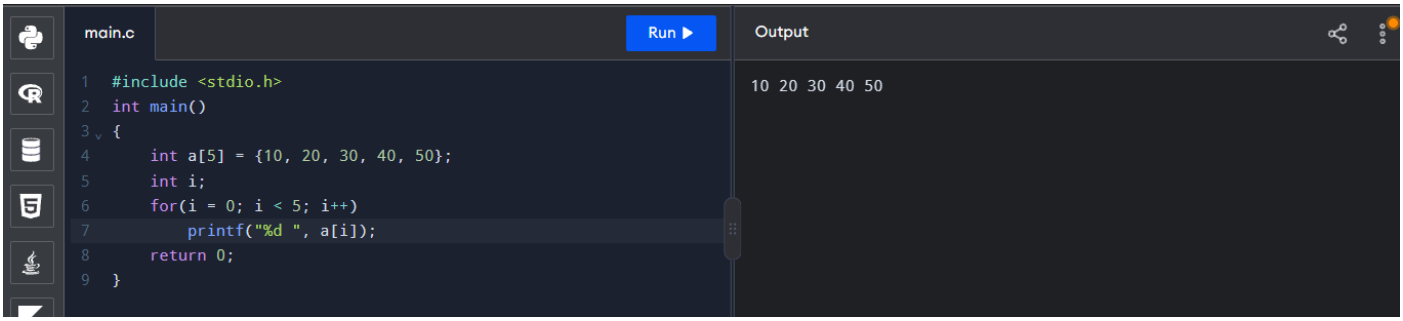
1. Array Traversal Program

Code:

```
#include <stdio.h>

int main()
{
   int a[5] = {10, 20, 30, 40, 50};
   int i;

   for(i = 0; i < 5; i++)
      printf("%d ", a[i]);
```




```
   return 0;
}
```

Output:

- Time Complexity: (O(n))
- Space Complexity: (O(1)) auxiliary space.

2. Sum of Array Elements

Code:

```
#include <stdio.h>

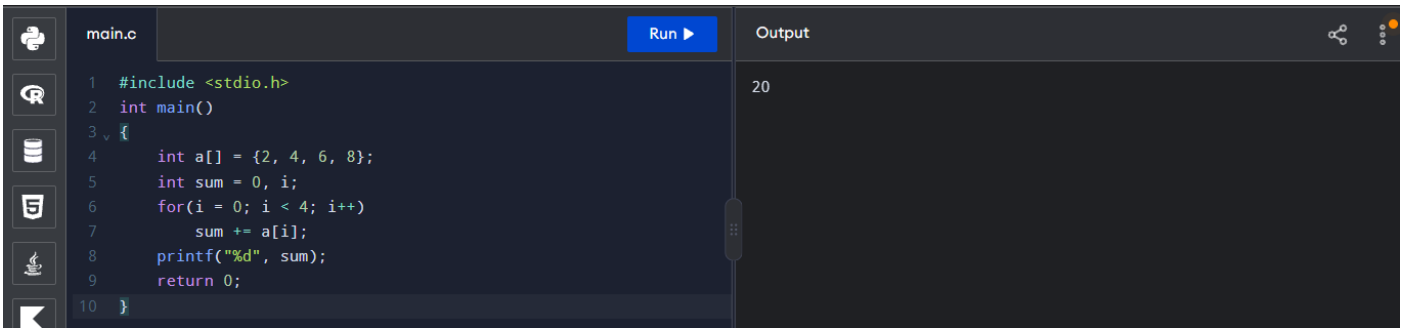
int main()
{
   int a[] = {2, 4, 6, 8};
```

```
int sum = 0, i;

    for(i = 0; i < 4; i++)
        sum += a[i];

    printf("%d", sum);
    return 0;
}
```

Output:



- Time Complexity: (O(n))
- Space Complexity: (O(1)) auxiliary space

3. Recursive Factorial Program

Code:

```
#include <stdio.h>

int fact(int n)
{
    if(n == 0)
        return 1;

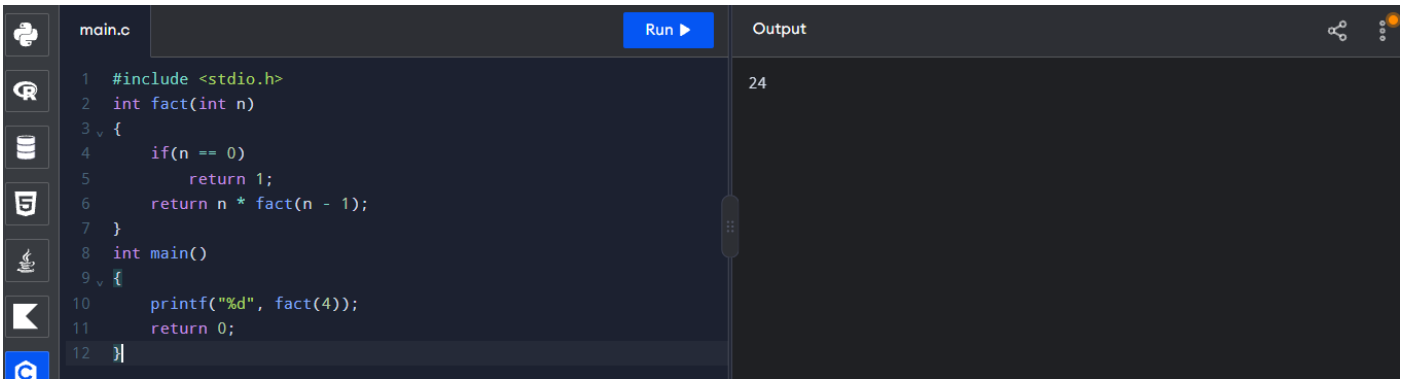
    return n * fact(n - 1);
}

int main()
{
    printf("%d", fact(4));
```

return 0;

}

Output:



- Time Complexity: (O(n))
- Space Complexity: (O(n)), due to recursive calls.

4. Recursive Fibonacci Program

Code:

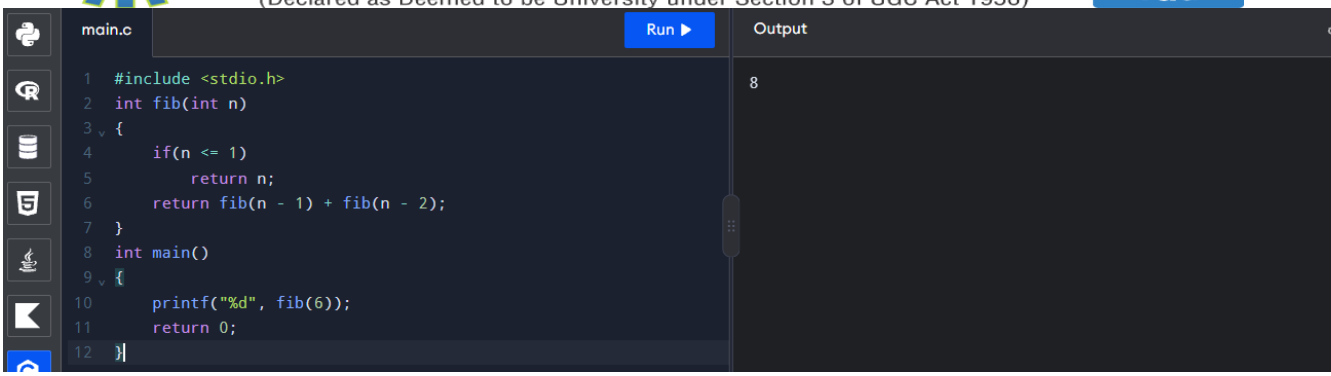
```
#include <stdio.h>

int fib(int n)
{
   if(n <= 1)
      return n;

   return fib(n - 1) + fib(n - 2);
}

int main()
{
   printf("%d", fib(6));
   return 0;
}
```

Output:



- Time Complexity: (O(2^n)) for this simple recursive implementation.
- Space Complexity: (O(n)), due to the maximum recursion depth.

5. Function as ADT Operation Program

Code:

```
#include <stdio.h>

int add(int x, int y)
{
   return x + y;
}

int main()
{
   printf("%d", add(15, 25));
   return 0;
}
```

Output:



- Time Complexity: (O(1))
- Space Complexity: (O(1))